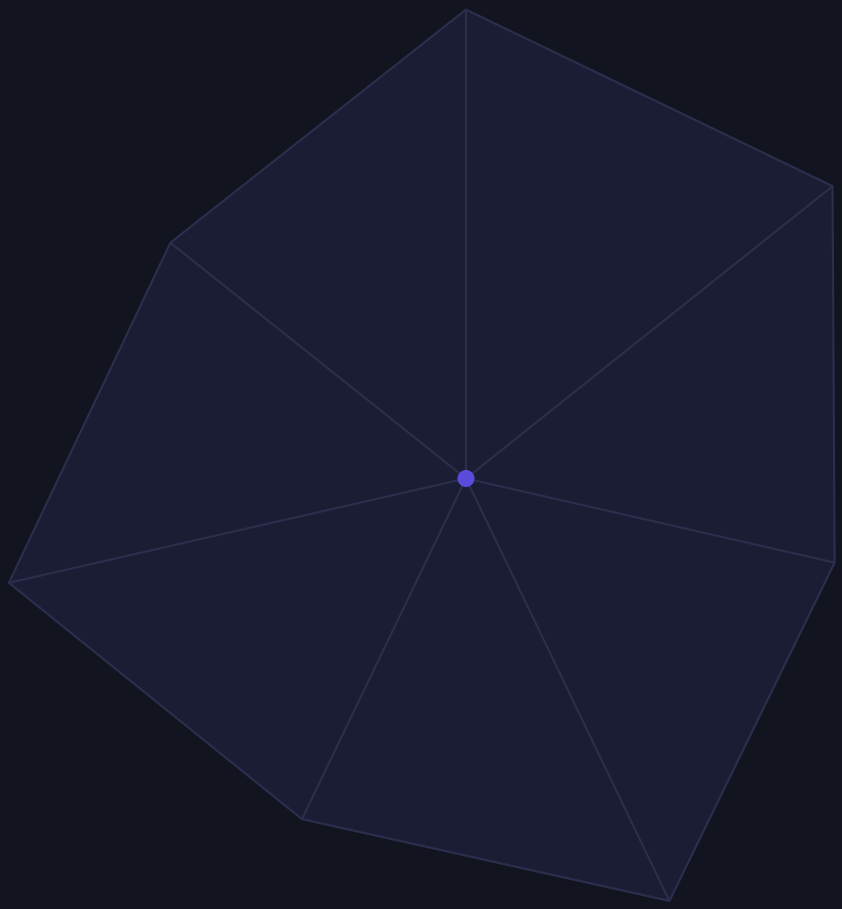


Obsidian Network

Technical Paper

The blockchain, its Proof of Time consensus, and every mechanism around it

Protocol 1.8.0 | Software 1.8.2 | 12 October 2026

Contents

1 Executive summary **4**

Design in one page . . . 4

2 System architecture **5**

Source of truth . . . 5

The four networks . . . 5

3 Cryptography and identity **6**

Wallet custody . . . 6

4 Data model: blocks, transactions and state **7**

Blocks . . . 7

Transactions . . . 7

State and the supply invariant . . . 7

Genesis . . . 8

5 Proof of Time consensus **9**

5.1 How time is established . . . 9

5.2 Who may produce a block . . . 9

5.3 Fork choice and reorganisation . . . 10

5.4 PoT Difficulty and Time-Rate . . . 10

6 Validators, finality and slashing **11**

6.1 Becoming a validator . . . 11

6.2 Checkpoint finality . . . 11

6.3 Equivocation and slashing . . . 11

6.4 Jailing . . . 12

7 Peer-to-peer networking **13**

7.1 Handshake and identity . . . 13

7.2 Limits . . . 13

7.3 Gossip and synchronisation . . . 13

7.4 Peer discovery . . . 13

8 Mining and issuance **14**

8.1 The mining gate . . . 14

8.2 Gate authority governance (protocol 1.8.0) . . . 14

8.3 Wallet-to-account policy . . . 15

9 Economics **16**

9.1 Issuance . . . 16

9.2 Prices . . . 16
9.3 ONS: the Obsidian Name Service . . . 16
9.4 Revenue split . . . 16
10 Node-runner rewards 17
10.1 Identity and registration . . . 17
10.2 Evidence . . . 17
10.3 Scoring and settlement . . . 17
11 The account platform and availability 18
Roles and automatic failover (1.8.1 line) . . . 18
12 Operations and observability 19
13 Security model, assumptions and limitations 20
13.1 Assumptions . . . 20
13.2 What is trusted . . . 20
13.3 Evidence behind the claims . . . 20
13.4 Design boundaries . . . 20
14 Release integrity 22
14.1 Release engineering . . . 22
A Appendix: parameters and identities 23
Genesis identities (1.8.0) . . . 23
Glossary . . . 23

1 Executive summary

Obsidian is a **Proof of Time (PoT)** blockchain with a fixed maximum supply of **21,000,000 OBS**. Its purpose is to let ordinary people, mostly on phones, take part in an open ledger: claim a small, protocol-defined mining reward every four hours, hold and send OBS from a wallet whose key they alone control, register `.obs` names, and run nodes that are paid from the protocol's only revenue source. Application data (names, balances, mining claims, validator bonds and the node-runner registry) is first-class chain state, not rows in a company database.

This paper describes the network as implemented in the repository: protocol **1.8.0** and software release **1.8.2**. Every number comes from the protocol parameters or the reference documentation.

Design in one page

Question	Answer
What is the consensus?	Proof of Time: who may produce a block is set by a deterministic validator schedule once protocol time has advanced. There is no hash race, no nonce and no difficulty gate.
Who produces blocks?	Bonded validators (exactly 20,000 OBS each), in a rotating schedule: `activeValidators[(height + round) mod n]`, 5-second slots.
How is finality reached?	Checkpoint finality: `floor(2N/3)+1` equal-weight validator signatures on a block. One validator, one vote; bond size adds no weight.
What is the clock?	The chain's own: median of the last 11 block timestamps. Device, website and CDN clocks cannot move it.
How is OBS issued?	Two paths only: a one-time 100,000 OBS genesis allocation, and mining claims. No premine, no administrator, no mint key.
What do users pay?	Gas of 0.02% of the amount (cap 0.01 OBS) to the Mining Pool; 0.05 OBS for a name. All prices are OBS-denominated and fixed by consensus.
Where does revenue go?	ONS fees only: 90% to the Node Runner Reward Pool, 10% to the treasury wallet designated by the first mining claim.
What stops mining Sybils?	A **gate certificate**: a short-lived signature from a committed issuer key, issued by the account platform to a signed-in account with one linked wallet. The issuer keys are changeable on chain by a majority of the current keys.
Who holds keys?	Users. Wallet keys are generated in the browser and never leave it. The node holds only its own identity key.

2 System architecture

The system is a monorepo of independent products that all reach the chain through the same node RPC. None of them is a second blockchain, and none holds keys or balances on the user's behalf.

Component	Role	Trust placed in it
Obsidian Core (node)	Validates and serves the chain; block production, P2P, mempool, RPC (default 8630), indexer, finality, state machine. The reference implementation.	Validating and serving chain data. Run your own to check it.
Interface (platform)	Account system, wallet-link proofs, mining certificate issuer, claim relay, explorer and sites. Roles: primary or edge (read-only).	Proxying to healthy nodes and holding account state. Not keys, funds, prices or consensus.
Web app / web wallet	Browser wallet UI (port 18790 / 18791 in testing); signs locally, submits through a node.	Generating and holding keys, signing. Its claims about the chain are re-checked by nodes.
Browser extension	The same wallet as a Chromium extension.	As the web wallet.
Desktop node app	Runs a node with a local dashboard.	As the node.
Cloudflare worker	Edge forwarding of read traffic.	Caching and forwarding bytes only; answers are labelled cached and expire in seconds.

Source of truth

```
consensus -> node state -> verified chain data -> APIs and indexes -> frontend cache
```

The arrow never points backwards. A website cannot create, move or destroy OBS; a CDN cannot change what a transaction does; a database is never the ledger; and the browser clock never decides eligibility, expiry or reward.

The four networks

Network	Chain id	Address prefix	RPC / P2P / interface	Purpose
mainnet	7777	obs	8630 / 8631 / 8788	Real value
testnet	7778	tobs	18630 / 18631 / 18788	Public rehearsal
staging	7779	sobs	28630 / 28631 / 28788	Release candidates
devnet	7780	dobs	38630 / 38631 / 38788	Local development, disposable

Networks are isolated by genesis document, genesis id, hash and parameters hash. A node rejects a block, transaction or peer from another network (`ERR_WRONG_NETWORK`, `ERR_WRONG_CHAIN_ID`) and refuses to start on a data directory written by a different network. Address prefixes differ per network, so a mainnet address cannot be mistyped into a devnet transaction.

3 Cryptography and identity

Purpose	Primitive
Transaction, block and node signatures	secp256k1 ECDSA, 64-byte compact `r||s`, low-S normalised
Address	`bech32m(hrp, ripemd160(sha256(compressed public key)))`, with a per-network prefix
Block ids, Merkle roots, state roots, PoT Weight	SHA-256
Wallet recovery phrase	24 BIP-39 words (256-bit entropy); BIP-32/BIP-44 path `m/44'/7777'/0'/0/index`
Browser vault	PBKDF2-SHA256 (600,000 iterations for new vaults), AES-256-GCM; passphrase at least 12 characters, NFKC-normalised
Node identity	A secp256k1 key generated on the node; `nodeId` is the 20-byte hash of its public key
Domain separation	Every signed message type has its own tag (for example `OBSIDIAN:MINING_GATE:v1`, `OBSIDIAN:GATE_AUTHORITY:v1`, `OBSIDIAN:NODE_REGISTRATION:v1`); a signature for one tag can never be a transaction

Hashing here proves integrity and identity. It is not a competition: computing faster earns nothing, which is the whole content of "PoT, not PoW".

Wallet custody

Keys are created in the browser from the WebCrypto CSPRNG and are never derived from an email, username or account id. No function in the wallet library sends key material over the network, and the stored vault holds only `{kdf, salt, iv, ciphertext}` plus public labels. The browser wallet builds transaction bodies with **the same encoder module the node uses**, so a signature produced in a browser verifies on a node. The 1.8.1 line additionally validates the vault envelope (format string, KDF cost between 100,000 and 10,000,000, salt, IV and ciphertext sizes) before any key derivation, so a hostile file cannot hang or confuse the wallet.

Known limit: the vault header (address label) is not authenticated data, so a locked page can be made to display another address; the signing key is unaffected. Fixing it needs a new vault version and a migration.

4 Data model: blocks, transactions and state

Blocks

Parameter	Value
Target interval	5 seconds
Maximum block size / transactions	2 MiB / 2,000
Soft / hard confirmation depth (guidance only)	12 / 64 blocks
Maximum ordinary reorganisation depth	256 blocks
Median-time-past window	11 blocks
Allowed future timestamp drift	60 seconds
Header nonce	**None.** There is no nonce field and no difficulty target in the header.

A header commits to height, parent hash, timestamp, cumulative PoT Weight, transactions root, events root, **state root**, parameters hash, protocol version, producer and the producer's signature. A block whose header fails is discarded; a block with one invalid transaction is rejected as a whole.

Transactions

`protocolVersion | chainId | sender | nonce | type | gas | body | validUntil | memo | signature`

Bodies use a deterministic binary codec (field map; varints; 128-bit little-endian big integers; length-prefixed UTF-8). Maximum transaction size 16 KiB, memo 256 bytes, 64 events per transaction. Replay protection is the pair (sender, nonce), the chain id in every signed transaction, `validUntil` (the protocol window is 240 blocks), and a transaction id that is a hash of the canonical bytes.

Id	Type	Purpose
1	PAYMENT	Transfer OBS; gas goes to the Mining Pool
2	ONS	Register, renew, transfer or update a `.obs` name
6	ORACLE	Informational OBS price observation (not used by core pricing)
7	MINING_CLAIM	Claim the protocol mining reward (needs a gate certificate)
8	VALIDATOR	Register with exactly 20,000 OBS bond, unbond, claim
9	TREASURY	Spend from the treasury (treasury wallet only)
10	GOVERNANCE	Protocol governance transactions
11	NODE_REGISTRY	Node-runner registration, heartbeats, attestations, fault reports
12	SLASH	Equivocation evidence; anyone may submit, no gas
13	GATE_AUTHORITY	Change the set of mining gate keys (approvals from a majority of current keys)

Ids 3, 4 and 5 belonged to discontinued features; no executor exists and the state machine rejects them as unknown types. An unknown type is always a hard rejection, never a silent no-op.

State and the supply invariant

State is a Merkleised map: accounts (balance, nonce, flags), the Mining Pool, treasury, ONS records, the Node Runner Reward Pool and registry, oracle submissions, validator records and the gate authority. Each block commits to a state root and an events root, and every node verifies both after applying the block. Applying a block is a pure function:

```
applyBlock(parentState, block) -> { state, events, receipts }
```

A candidate state is built on a copy and committed only after the state root matches, so an invalid block never alters state. The supply invariant is checked on every block:

```
sum(balances) + mining pool + locked validator and node bonds = metrics.totalSupply
metrics.totalSupply <= 21,000,000 OBS
```

If it ever fails, `/health` returns an error. There is no administrative mint and no path past the cap.

Genesis

The genesis allocation is one grant of **100,000 OBS**, awarded atomically to the first valid mining claim on a chain. That wallet becomes the on-chain treasury wallet. It is not paid at registration, not per account and not to a founder address. A new interface account starts with 0 OBS. Genesis ids are derived deterministically from the genesis document and parameters (network, chain id, protocol version, timestamp, address prefix, committed validator keys and committed gate keys), so two nodes agree on the id only if they carry the same lists.

5 Proof of Time consensus

Obsidian's core product is a claim a person makes from a phone every four hours. That design has exactly one hard problem: **deciding what time it is without letting the participant decide.** Proof of Work sidesteps the problem by making production expensive. Obsidian solves it directly: the chain carries its own clock, and every rule that matters (claim windows, name expiry, reward periods, transaction expiry) is evaluated against it.

	Proof of Work	Obsidian Proof of Time
Right to produce a block	Computation spent in a race	The validator schedule, once protocol time has advanced
What an attacker must acquire	Hash power	A majority of the validator set; they still cannot move time
Headline metric	Hash rate	**Time-Rate**: verified blocks and transactions per minute of protocol time
"Difficulty"	Hashes per block	**PoT Difficulty**: how block spacing compares with the 5-second target (a measurement, not a gate)
Fork choice	Most accumulated work	Finalized anchor, fixed PoT Weight, height, lowest hash
Energy	Security grows with it	Irrelevant: a faster machine buys nothing

5.1 How time is established

The clock is the **median** timestamp of the last 11 blocks. A block is accepted only if all three conditions hold:

```
timestamp >  median time past of the last 11 ancestors
timestamp >  parent timestamp                      (strict monotonicity)
timestamp <= this node's own clock + 60 seconds    (maxFutureDriftSeconds)
```

The first two use chain data only, so every node reaches the same verdict. The third is the only place a node's own clock appears, and it can only reject a block claiming the future, never admit one the chain rules reject. A node with a wrong clock therefore harms only itself.

```
protocolTime = max(local wall clock, head.timestamp + 1, medianTimePast + 1)
```

A node never believes it is earlier than the chain it holds, so back-dating cannot bypass a cooldown. Mining claims, name terms and transaction expiry all read this clock, and cooldowns are enforced by validators against block timestamps, never by an interface countdown.

5.2 Who may produce a block

```
round(parent, block)  = max(0, floor((block.timestamp - parent.timestamp) / 5) - 1)
proposer(height, round) = activeValidators[(height + round) mod activeValidatorCount]
```

The active set is sorted by address with jailed validators excluded. Round 0 is the ordinary case: the scheduled validator has the first ten seconds after its parent. Each further whole slot hands the turn to the next validator. The round is derived from two timestamps already committed in headers, so no extra header field or negotiation is needed. There is no randomness to grind, no leader election to win and no advantage to computing faster. A block produced out of turn is rejected (`ERR_NOT_PRODUCER_TURN`).

Two modes, one committed indicator. The consensus value `validatorModeEstablished` is committed in the state root and is set only by the first successful validator registration.

Mode	Indicator	Who may propose
Bootstrap	false	Any node (this is how the first validator arrives)
Established	true, for ever	The active set only. An empty active set **halts** the chain; it resumes only when a jail term lapses or a new validator registers with a full bond. No operator action, no admin call, no permissionless fallback.

This deliberately chooses validator-set safety over permissionless fallback liveness.

5.3 Fork choice and reorganisation

A candidate is considered only if it is protocol-valid and descends from the node's locally verified finalized checkpoint. Eligible tips are compared in order:

- greatest accumulated fixed **PoT Weight** (every valid block adds one unit; fees and transactions add none);
- greatest height;
- lowest header hash, a total order over otherwise equal candidates, so every node resolves the same two branches identically.

There is no arrival-order component and no stake weighting. Ordinary reorganisation finds the common ancestor, replays the winning branch, recomputes state roots and revalidates the mempool; it is capped at 256 blocks and can never cross the locally finalized height. A valid finality certificate may move the chain onto its certified branch beyond the ordinary cap, but only after signatures, committee, ancestry and full block validation pass.

Because weight is per block, fork choice alone gives no economic protection against someone who can produce valid blocks. The protection is the proposer schedule, the reorganisation cap and finality.

5.4 PoT Difficulty and Time-Rate

PoT Difficulty is computed from the last 720 blocks by comparing observed spacing with the 5-second target and is bounded to 25%-400%. The protocol does not reject a block for being faster than target: a syncing node and a burst of activity are legitimately faster. Time-Rate (24-hour window) is the participation metric an explorer shows where a PoW chain shows hashrate.

6 Validators, finality and slashing

6.1 Becoming a validator

Any account may register with a `VALIDATOR` transaction carrying a bond of **exactly 20,000 OBS**, a 33-byte compressed public key and a declarative commission. The bond leaves the spendable balance but is still the validator's property and counts in the supply invariant. Unregistering starts a delay of **20,160 blocks** (about 28 hours); only then can the bond be claimed.

6.2 Checkpoint finality

For a target block, every active, positively bonded validator in the target's parent state has one membership vote. Bond size is admission collateral only and adds no weight. A checkpoint needs `floor(2N/3) + 1` distinct valid signatures. A vote commits to the protocol and network identity, the previous finalized checkpoint, target height, hash and parent, proposer round and validator-set hash.

Safety claim. Two conflicting certified checkpoints require more than one third of the committee to have signed both, a provable offence, or a cryptographic failure. The quorum arithmetic is tested for every committee size from 1 to 3,000.

Network	Committed bootstrap keys	Quorum	Fault tolerance
mainnet	4	3	One validator may be offline
testnet	3	3	**None**: one offline validator stops finality
staging, devnet	none (operator supplies its own)	-	-

The bootstrap committee is a public list hashed into the genesis id, so it is not a local opinion and a peer cannot talk a node into another list. Before the first checkpoint only the committed keys may vote and only if each has registered with the exact bond. A network with an empty set has no bootstrap committee: it finalizes nothing until an on-chain validator set has been stable for 64 parent states. Nothing is ever derived from the fork a node happens to hold.

6.3 Equivocation and slashing

A validator forfeits **half its bond** (10,000 of 20,000 OBS) if it is proven, with signatures every node can check, to have signed two different headers for the same height and round, or two conflicting finality votes for one anchor. Everything else is not slashable: being offline, missing a slot, restarting, losing connectivity, or voting twice for the same block. Liveness costs time (the jail), never capital.

- Enforced by a `SLASH` transaction any account may submit, with no gas and no reward to the submitter; the offender, amount and destination are all derived.
- The slashed amount moves to the Mining Pool in the same transition; total supply is unchanged.
- The offender leaves the proposer rotation and the committee in that block, and one registration is slashed once (evidence ids are committed in the state root).
- The evidence window is the unbonding delay, 20,160 blocks.
- A node never produces a slashable pair by itself: a proposer's slot is written to a crash-safe local lock before the signature exists.

- Evidence is bounded: at most 8 reports and 65,536 bytes per block, 32 reports and 1 MiB per producer attempt, 256 pending in the mempool.

6.4 Jailing

When a block is produced in a later round than 0, the round-0 proposer of that height is recorded as having missed its slot. Each recorded miss adds 1 to its counter; a block it produces subtracts 1 (floor 0). Crossing **40** net misses jails the validator for **10,080 slots** (50,400 seconds, about 14 hours). The term is a duration of protocol time committed in the state root, so it expires even if the chain has stopped. Jailing is automatic: no dashboard, no administrator and no vote.

7 Peer-to-peer networking

7.1 Handshake and identity

Nodes connect over WebSocket with JSON envelopes. The acceptor speaks first with a fresh **challenge**. The dialer's `hello` carries node id, network id, chain id, genesis id, parameters hash, versions, chain tip and finalized checkpoint, signed with the node identity key over that challenge; the acceptor's `hello_ack` is signed over the dialer's nonce. A recorded handshake is therefore useless on any other connection. Peers that differ in genesis id, network, chain id, parameters hash or protocol version are refused before any block is exchanged, as are peers below `MIN_CORE_VERSION`. A peer on the previous patch release with the same protocol is accepted. Before the handshake completes nothing larger than 64 KiB is read.

7.2 Limits

- Token-bucket budgets for control messages, blocks and transactions on every link; a link that runs out of credit is dropped and its address refused for ten minutes.
- A block batch is accepted only as the answer to a request this node made.
- One remote address may hold eight inbound connections (loopback exempt). A peer is known by the address it connects from, never by the host it claims.
- Dedup sets, pending queues, candidate counts and signature work are bounded; transactions and finality objects are validated before relay.

7.3 Gossip and synchronisation

Blocks, transactions, finality votes, certificates and validated equivocation evidence are pushed to peers. `ping/pong/status` messages carry the sender's tip. A node that sees a better tip under the fork choice, or an announced block whose parent it lacks, requests blocks with `getblocks {from, limit, id}`. The first request starts a little behind the node's own head; that overlap doubles until the common ancestor is found, so a node that spent minutes on its own fork rejoins. After that a cursor pages through the peer's chain in batches of up to 128 blocks. Blocks are verified before they are applied; a peer that claims a better chain but delivers nothing new stops being believed.

7.4 Peer discovery

A node starts from the addresses it is given (`--seeds`, `OBSIDIAN_SEED_NODES`, or `join --peer` with a pinned genesis id). Once connected it learns more peers from gossiped peer lists, saves them in `peers.json` and reuses them after a restart. Seeds are retried every 15 seconds, so a seed that was offline at start is found when it comes up. Learned addresses are validated, and every peer is checked against the node's genesis id, protocol version and finalized checkpoint at the handshake.

8 Mining and issuance

Mining on Obsidian is a **protocol claim**, not a browser game. The node decides whether a wallet is eligible, how much the claim pays and which claim id is next. The browser only signs and submits.

Parameter	Value
Minimum interval between claims by one wallet	4 hours
Maximum claims per wallet per rolling day	6
Network daily reward at genesis participation	0.001 OBS
Reward per claim at genesis participation	0.000166666666666666 OBS
Reduction	0.5% (50 bps) per 100,000 active miners
Absolute daily floor	0.0002 OBS
Active-miner window	30 days with at least one valid claim
Claims per wallet per block	1

Eligibility, amount and claim id are derived from chain data (`/mining/schedule`, `/mining/status`). Nothing in the mining path reads a wall clock, request header or client timestamp. Rewards are settled with Mining Pool funds first, and only the remainder is newly issued, so gas recycled into the pool pays rewards before any new OBS is created.

8.1 The mining gate

Since 1.7.0 a `MINING_CLAIM` is valid only with a **gate certificate**: a signature, by an active issuer key, over `GATE|networkId|chainId|address|claimId|issuedAt`. The platform issues it to a signed-in, MFA-confirmed account whose one linked wallet is that address. The certificate is valid from 60 seconds before to 15 minutes after `issuedAt`, measured against the block that includes the claim, for one claim id only. The chain still decides timing and eligibility from the wallet's own history; the gate only decides whether the claim may be attempted.

Purpose: Sybil resistance, so that one person cannot create unlimited wallets and claim for each.

8.2 Gate authority governance (protocol 1.8.0)

The active gate keys are chain state, changeable by a `GATE_AUTHORITY` transaction without a new genesis:

- Valid only if its epoch is exactly the current epoch + 1, so an old approval cannot be replayed.
- Needs approvals from at least `floor(n/2) + 1` of the n current active keys, each signing network, chain, epoch and the exact change (domain `OBSIDIAN:GATE_AUTHORITY:v1`).
- At most 8 keys added and 8 removed per update; the result has between 1 and 8 active keys; a revoked key can never be re-added.
- A revoked key's certificates stop working at the first block after the update, even inside their 15-minute life.
- Private keys are never broadcast, put on chain or printed. The transaction's sender decides nothing; authority comes only from the approvals.

Active keys	Approvals	One stolen key can...	Lost keys
1	1	**Everything**, including replacing itself	Losing it ends the gate: new genesis
2	2	Sign certificates until revoked; cannot rotate alone	Losing either blocks rotation
3	2	Sign certificates until the other two revoke it	One lost or stolen: recoverable
5	3	As above, wider margin	Two lost or stolen: recoverable

Mainnet `init --generate-gate-key` refuses fewer than 3 keys unless `--allow-single-gate-key` is passed; readiness advisories report a closed, single-key or sub-three-key gate.

8.3 Wallet-to-account policy

- **One wallet per account.** Linking requires a signature on a one-time challenge (5 minutes) made by the wallet key, and is permanent. A different wallet is refused (`ERR_WALLET_LOCKED`).
- **One account per wallet.** A wallet already linked elsewhere is refused (`ERR_WALLET_TAKEN`).
- The wallet is an independent key, never derived from account information.
- Eligibility is decided by consensus from the wallet's last claim; the platform imposes no rules of its own.
- Consequence: losing the wallet key means losing mining with that account, because the platform never holds the key.

9 Economics

The protocol has no sale, no premine, no administrator and no mint authority. Every price is denominated in OBS and fixed by consensus, so no external feed can change what anything costs.

9.1 Issuance

Path	Amount	When
Genesis allocation	Exactly 100,000 OBS, once, ever	To the wallet behind the first protocol-valid mining claim; it becomes the treasury
Mining rewards	Per schedule in section 8	Per accepted claim, capped by the 21,000,000 OBS supply cap

9.2 Prices

Action	Price	Destination
Gas on a value-bearing transaction	min(0.02% of the amount, 0.01 OBS)	Mining Pool
`.obs` name registration or renewal	0.05 OBS	ONS revenue, split 90/10
Validator bond	Exactly 20,000 OBS (returned in full at unbonding; half forfeited on provable equivocation)	Locked, not spent
Node runner registration	Nothing	No transfer exists

Gas is deterministic: the node's quote and the wallet library's `expectedGas` return the same number, and a transaction declaring less is rejected (`ERR_BAD_GAS`). Gas is never ONS revenue and never goes to a company or a founder address.

9.3 ONS: the Obsidian Name Service

A `.obs` name maps to exactly one wallet address, and that mapping is blockchain state with no DNS, registrar or company in the middle. Names are 3-63 characters, registered for 365 days with a 30-day grace period. Only the current owner may transfer a name; owner and address move together; a transfer costs only standard gas.

9.4 Revenue split

```
ONS revenue
   |-- 90%  ->  Node Runner Reward Pool
   `-- 10%  ->  protocol treasury wallet (designated by the genesis rule)
```

The arithmetic is exact integer math: `nodePool = floor(total x 9000 / 10000)`, `treasury = total - nodePool`, so the parts always add back to the whole and no rounding dust is lost. The split happens inside the state transition on every node. The treasury share is paid in the same block as the fee once a treasury exists; before that it is held as an explicit unclaimed obligation. The node share accumulates in the pool and is paid once per 24-hour period (section 10).

10 Node-runner rewards

Independent nodes are paid from the 90% pool, with no admin panel, allowlist or company database. **The protocol never asks a node how it did**: there is no field in which a node can report its own uptime, participation or efficiency.

10.1 Identity and registration

```
Node Identity (secp256k1, generated on the node)  ->  Node Operator (human)  ->  Reward Wallet (obs1... address)
```

A `NODE_REGISTRY` registration needs **two signatures**: the node key signs a statement (network, chain, node id, reward wallet, issuedAt, expiresAt, capped at one hour) and the reward wallet signs and pays for the transaction. Someone who steals a node key cannot redirect rewards, and someone who knows your wallet address cannot attach it to their node. Registration locks no funds. One wallet may back exactly one node. An IP address is never an identity.

10.2 Evidence

Statement	Signed by	Proves
HEARTBEAT	The node key	This node is alive in this period at this height (one per period)
ATTEST	An observing node	I observed another node alive (a node cannot attest itself)
REPORT_FAULT	An observing node	Another node misbehaved; counts only when independent reporters corroborate

Blocks produced are read from the chain, not from any claim.

10.3 Scoring and settlement

```
scoreBps = uptime x 40% + participation x 25% + reliability x 20% + responsiveness x 15%
weight   = scoreBps - minScoreBps   (registered, uptime >= 50%, score >= 10%)
share    = min(5%, weight / sum(weights))
```

Uptime requires independent attestation: a node that heartbeats perfectly but is corroborated by nobody scores zero uptime. Latency is not a component, so expensive hardware earns nothing extra. Periods are fixed 24-hour windows of protocol time; the first block of a new period settles the one that closed, on every node, with no operator or cron job. Any single node is capped at 5% of a period; the remainder is carried forward, never burned and never sent to the treasury. A reward wallet can change only through the node key plus the current wallet, effective one period later, so a stolen node key cannot sweep a period in flight.

11 The account platform and availability

The interface server holds accounts (with sign-in and a second factor), wallet-link proofs, invite state and mining certificate issuance. It never holds a wallet key. It reads nodes over HTTP and fails over between them. Sessions use an HttpOnly, SameSite=Lax cookie (Secure behind a trusted proxy), 14-day lifetime, a strict content-security policy, CSRF and same-origin checks, and (since 1.8.1) a fresh session identifier at sign-in so a planted cookie is never a credential.

Roles and automatic failover (1.8.1 line)

Several primary-capable hosts can share one durable account store. They elect a leader with a **lease** that carries a monotonically increasing **fencing token**:

- The leader trusts itself for `ttl - 2 x skew`; a follower takes over only after the lease expires plus the skew allowance.
- Every store write is fenced: a deposed leader's write is refused, so a paused or partitioned former leader cannot issue certificates or accounts.
- A host without the lease is a standby and answers 503 `ERR_NOT_LEADER` (fails closed). An edge host is read-only and refuses account, wallet-link and certificate requests.
- Sessions flushed to the shared store (about 2 seconds) survive a failover.

> **Deployment note**
>
> The lease is a file. It is safe on one host or a correct shared POSIX file system, **not on NFS** and most network file systems. Real multi-site high availability needs a coordination service or database with compare-and-set. This is application-level failover, not infrastructure availability. Every primary-capable host also needs its own copy of the gate keystore.

12 Operations and observability

Health endpoints distinguish what is wrong, so a load balancer and an operator can act differently:

Endpoint	Meaning
`/livez`	The process is alive
`/readyz`	`status`: ready, ready-degraded or not-ready; per-dimension checks: chain, sync, p2p, rpc, identity, finality (max lag 64), gate authority, last block age. HTTP 200/503 is decided only by chain, sync and p2p. `advisories` (for example a single gate key) never change the status.
`/health`, `/supply`	Supply invariant and chain summary
`/params`, `/pot`, `/validators`, `/finality`, `/gate`	Consensus identity, PoT metrics, validators, checkpoint, gate keys and epoch
`/audit/compliance`	Confirms removed mechanisms (proof of work, header nonce, admin minting, KYC-gated mining, native exchange) are absent

Node data directories are protected by a lock file (a second process refuses to start; a stale lock from a crashed process is taken over safely, with the 1.8.2 fix for a simultaneous-start race). `init` is idempotent and never overwrites an existing genesis; `--require-init` forbids creating a chain implicitly; `join` adopts a network only with a pinned genesis id. The gate CLI supports `gate status|propose|sign|submit` with a dry run. Disaster-recovery procedures for gate keys are in the gate recovery runbook.

13 Security model, assumptions and limitations

13.1 Assumptions

Assumption	If it fails
Clocks are loosely synchronised (drift bound 60 s; rounds derive from timestamps)	Rounds and acceptance disagree; honest nodes with bad clocks fall behind
Fewer than one third of the committee is Byzantine	Two conflicting checkpoints can be certified; the offence is provable and slashable, but already done
At least `floor(2N/3)+1` committee members are honest and online	No new checkpoints; the chain does not finalize wrongly, it stops finalizing
Bond is the only validator Sybil resistance	k bonds buy k votes; the protocol cannot tell one operator with ten seats from ten operators
A majority of gate keys is honest and available	A stolen majority can issue certificates and rotate keys; a lost majority needs a new genesis
The genesis id an operator pinned is the right one	Joining the wrong network; nothing in the protocol can tell
secp256k1 and SHA-256 hold; domain tags are never reused	Everything

13.2 What is trusted

Component	Trusted for	Not trusted for
Core node	Validating and serving chain data	A malicious node can lie about state but cannot forge a signature; compare against your own node
Interface server	Proxying, account and invite state	Keys, funds, prices, balances, consensus
Browser	Generating keys and signing	Anything it asserts about the chain
CDN / Cloudflare	Caching and forwarding bytes	Being an authority
Oracle feed	Informational telemetry	Pricing: it is unpermissioned with no Sybil resistance

13.3 Evidence behind the claims

The 1.8.1 line was accompanied by new adversarial and property tests: quorum arithmetic for N from 1 to 3,000; two independent nodes applying the same blocks reach the same state; invalid blocks leave state unchanged; conflicting checkpoints refused; gate rotation, revocation, replay, cross-network approvals and quorum loss; interface failover with real processes (kill, restart, pause and resume, partition); a stress harness that exposed and then closed a data-directory lock race (2 of 40 trials before the fix, 0 of 80 after). A secret scanner, dependency audit (0 vulnerabilities in packages with dependencies) and deterministic builds are part of the pipeline. The reference consensus documentation names the code and test that back each rule.

13.4 Design boundaries

- **Nothing on chain is private.** Masked addresses are a measure, not a guarantee.

- **Reorganisation before finality is possible;** the 12/64 depth figures are guidance, not finality. Ordinary reorganisation deeper than 256 blocks is refused.
- **Genesis-open mode** leaves a brand-new network unprotected by the schedule until validators register.
- **Testnet's three-validator committee has no fault tolerance** (quorum 3 of 3).
- **Equal-weight validators:** a fixed bond is the barrier to entry; a stake-weighted design would be a separate protocol change.
- **A weak wallet passphrase is a weak wallet**; the vault header is not authenticated.
- **The oracle is informational.**

14 Release integrity

14.1 Release engineering

Release archives are built deterministically: sorted archive entries, a fixed modification time taken from the source commit, numeric owner 0, `gzip -n`, and normalised SBOMs. For release 1.8.2, a second build of the same commit on the same machine and toolchain produced byte-identical archives (9 zip, 10 tar.gz) and SBOMs, and the repository records this in a reproduction record.

A release tool (`scripts/release-tool.mjs`) signs and verifies with Ed25519 over the version, commit and the hashes of the checksum list and manifest, checks that every version in the release agrees, and accepts authorship only from a publisher fingerprint the verifier pinned. A valid signature from an unpinned key is reported as signed but untrusted.

A Appendix: parameters and identities

Parameter	Value
Maximum supply	21,000,000 OBS
Genesis allocation	100,000 OBS to the first valid claim
Block interval / size / transactions	5 s / 2 MiB / 2,000
Maximum reorganisation / median window / drift	256 blocks / 11 blocks / 60 s
Finality quorum	floor(2N/3)+1 equal-weight votes
Validator bond / unbonding	20,000 OBS / 20,160 blocks (about 28 h)
Jail threshold / term	40 net missed slots / 10,080 slots (about 14 h)
Equivocation penalty	50% of the bond (10,000 OBS), to the Mining Pool
Mining interval / daily cap / reward	4 h / 6 claims / 0.000166666666666666 OBS per claim at genesis participation
Miner reduction / floor	0.5% per 100,000 active miners / 0.0002 OBS per day
Gas	min(0.02% of amount, 0.01 OBS)
ONS fee / term / grace	0.05 OBS / 365 days / 30 days
Revenue split	90% node runners / 10% treasury
Node score weights	uptime 40, participation 25, reliability 20, responsiveness 15 (percent)
Node cap / minimum uptime / minimum score	5% of a period / 50% / 10%
Gate certificate window	60 s before to 15 min after issuedAt, one claim id
Gate update threshold	floor(n/2)+1 of the current active keys; 1-8 keys; at most 8 changes per direction
Wire limits	64 KiB before handshake; 8 MiB payload; 128 blocks per batch; 8 inbound per remote address
Parameters hash (1.8.0)	1d6cba55f0975f8b0d6345658f1c1cb1

Genesis identities (1.8.0)

Network	Genesis id
mainnet	3722ae03f8e74d1ebd5f0dc70addc5122d7d7c70
testnet	7a5b20e504d6c0a278bf23799a7394d17470856f
staging	3c4461cff67c0306d97ff8de150dcb8937eb2a5a
devnet	9e56e977b443aec428f3b59ce4880062250e7236

Always pin the genesis id from a source you trust (for example by running `genesis init --network <name>` from a verified release) before joining a network. A gate-key list is part of the genesis id, so a deployment that created its own gate keys has a different id.

Glossary

Term	Meaning
PoT Weight	A fixed unit per valid block, accumulated for fork choice; not work and not stake

Term	Meaning
Protocol time	The chain-derived clock every rule uses
Checkpoint / certificate	A block endorsed by a quorum of the finality committee; nodes never reorganise below it
Equivocation	Signing two conflicting messages in the same protocol context; the one slashable offence
Gate certificate	A short-lived signed permission to attempt one mining claim
Epoch (gate)	A counter incremented by each accepted gate key update
Seal	The smallest unit of OBS (1e-18)
Edge / primary	Read-only versus account-serving interface roles